

TRABALHO DISCENTE EFETIVO TDE 2

Código Morse com árvore binária e Flood Fill com estrutura linear

Semestre	2026 segundo semestre	Carga horária	14 horas
Modalidade	Trio ou quarteto	Entrega	Até 08/10/2026

1 Objetivo

Desenvolver duas aplicações exclusivamente em Java ou C que relacionem estruturas de dados a problemas práticos. Não serão aceitas outras linguagens, pois abstrações mais sofisticadas podem afastar a atividade do escopo pedagógico, especialmente da compreensão de valores, endereços e referências. A primeira aplicação utilizará uma árvore binária para representar o Código Morse. A segunda implementará o Flood Fill de duas formas: uma versão controlada por pilha própria e outra controlada por fila própria.

Cada projeto deverá possuir seu próprio menu de navegação. As operações poderão ser repetidas e o usuário deverá conseguir retornar ao menu sem reiniciar a aplicação.

2 Organização das equipes

- O trabalho poderá ser realizado em **trios ou quartetos** (apenas).
- Entrega individual somente será aceita com autorização prévia do professor.
- Todos os integrantes deverão compreender e saber modificar o projeto.

3 **Parte 1** Código Morse com árvore binária

Construa dinamicamente uma árvore binária que represente letras e números do Código Morse. A partir da raiz, cada ponto (.) conduz ao filho esquerdo e cada traço (-) conduz ao filho direito. A posição alcançada pela sequência armazena o caractere correspondente.

3.1 Funcionalidades obrigatórias

- construir a árvore a partir de uma tabela de símbolos Morse. No código, a equipe deverá chamar manualmente as operações de inserção, de modo que a árvore resultante seja equivalente à tabela fornecida;
- codificar texto comum para Código Morse: a função recebe texto legível e imprime a sequência correspondente em Morse;
- decodificar sequências Morse para texto: a função recebe uma entrada em Morse e imprime a saída em texto legível;
- codificar o conteúdo de um arquivo .txt que contenha texto legível; decodificar o conteúdo de um arquivo .txt que contenha uma única linha em Código Morse, conforme o formato definido nesta atividade;
- tratar espaços entre letras, palavras e símbolos inválidos;
- exibir a árvore por meio de um **diagrama legível, no console gráfico** ou em **interface gráfica (com lib externa se necessário, apenas para gráficos)**.

3.2 Símbolos, separadores e exemplos

Na decodificação, tanto pelo menu quanto por arquivo, a entrada deverá conter uma única linha formada exclusivamente pelos caracteres ponto (.), traço (-), barra (/) e espaço. Não serão aceitos quebra de linha, tabulação, letras, números ou qualquer outro símbolo. Os pontos e traços que formam o código de uma mesma letra devem permanecer juntos. Um espaço separa o código de uma letra do código da letra seguinte.

Neste trabalho, o caractere barra (/) foi escolhido para representar o espaço entre palavras. Durante a codificação, cada espaço entre palavras deverá gerar uma barra. Durante a decodificação, cada barra deverá voltar a ser um espaço. A barra é um separador definido para a representação textual desta atividade e não pertence ao código de nenhuma letra.

Exemplos de **codificação**:

Entrada: SOS

Saída: ... --- ...

Entrada: OLA MUNDO

Saída: --- .-. .- / -- .- .- .- ---

Exemplos de **decodificação**:

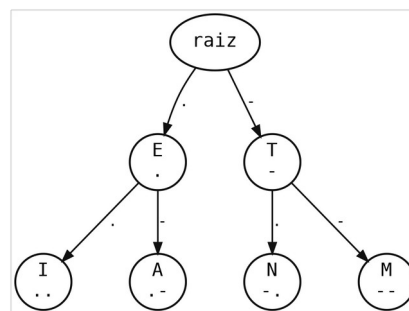
Entrada: ... --- ...

Saída: SOS

Entrada: --- .-. .- / -- .- .- .- ---

Saída: OLA MUNDO

A visualização não precisa representar uma árvore binária de busca. A posição dos nós é definida pelos símbolos do Código Morse. Devem ficar claras a raiz, as ligações e as direções correspondentes a ponto e traço.



3.3 Organização sugerida

- MorseNode representa um nó e suas referências esquerda e direita.
-
- FileService lê o arquivo de texto para codificação e o arquivo Morse para decodificação.
- Main apresenta o menu e coordena as chamadas.

4 Parte 2 Flood Fill com pilha e fila

Implemente o algoritmo Flood Fill, semelhante à ferramenta balde de tinta. O programa receberá uma imagem, uma coordenada inicial e uma nova cor. A região conectada que possuir a mesma cor do pixel inicial deverá receber a nova cor.

A busca deverá considerar somente quatro vizinhos: cima, baixo, esquerda e direita. A equipe deverá implementar as duas soluções: uma utilizando pilha própria e outra utilizando fila própria. Não utilize bibliotecas nem implementações prontas de pilha e fila. Em Java, isso inclui Stack, Queue e Deque; em C, inclui estruturas equivalentes obtidas de bibliotecas externas. O controle principal deverá ser iterativo e não depender de recursão.

4.1 Funcionamento esperado

- abrir uma imagem selecionada pelo usuário;
- receber a **coordenada inicial** e a nova cor;
- armazenar a cor original do pixel inicial;
- verificar limites antes de acessar um pixel;
- pintar posições válidas e adicionar seus quatro vizinhos;

- salvar as etapas em arquivos de imagem, preferencialmente no formato BMP. PNG também será aceito, desde que a equipe mantenha o mesmo formato durante toda a execução (por exemplo: passo_0001.bmp, passo_0002.bmp etc.).

4.2 Funcionamento em alto nível

A pilha e a fila armazenam posições da imagem, formadas por linha e coluna. O programa coloca a posição inicial na estrutura e entra em um laço. A cada repetição, retira uma posição. Se ela estiver fora da imagem, será ignorada. Se a cor atual for diferente da cor original da região, também será ignorada.

Quando a posição ainda possui a cor original, o programa troca sua cor e insere as coordenadas dos quatro vizinhos. A mudança de cor funciona como memória de visitação. Se a mesma posição aparecer novamente, sua cor já terá mudado e ela será descartada. Dessa forma, não é necessário manter uma lista separada de visitados ou um dirty bit. Se a nova cor for igual à cor original, o algoritmo deverá encerrar antes de iniciar.

A versão com pilha tende a avançar por caminhos mais profundos. A versão com fila tende a preencher a região em camadas. A ordem visual será diferente, mas as duas versões deverão preencher a mesma região final.

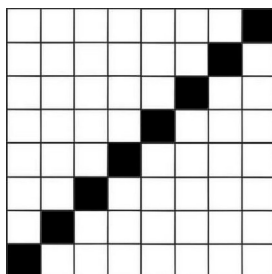
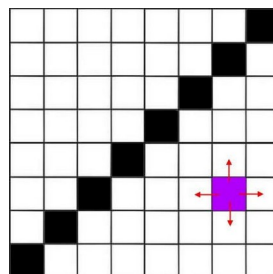
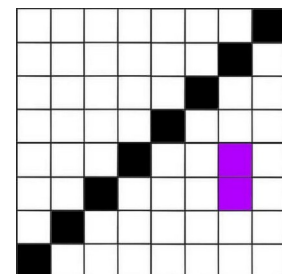


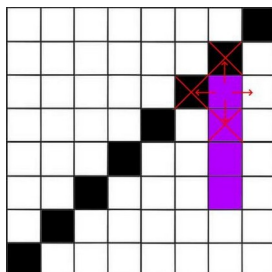
Imagem inicial



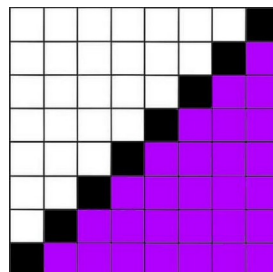
Coordenada inicial



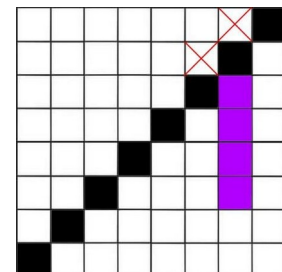
Primeiros pixels



Vizinhos examinados



Região preenchida



Posições ignoradas

A execução deve permitir observar o preenchimento acontecendo. Em Java, a equipe poderá utilizar Swing, JavaFX ou outra biblioteca apenas para a interface gráfica. Em C, poderá utilizar uma biblioteca apenas para abrir, exibir e salvar imagens. As estruturas de pilha e fila e o algoritmo Flood Fill deverão ser implementados pela equipe.

Sem interface gráfica, o programa deverá gerar imagens numeradas em BMP ou PNG, mantendo um único formato. Para imagens grandes, a equipe poderá salvar um quadro a cada quantidade fixa de pixels. Devem existir a imagem original, etapas intermediárias e a imagem final.

4.4 Organização sugerida

- Position armazena linha e coluna.
- Node, Stack e Queue implementam as duas **estruturas** lineares **próprias**.
- FloodFill executa o algoritmo.
- ImageService abre, salva e registra as etapas.

5 Menus dos projetos

Código Morse

- 1 - Codificar texto digitado
- 2 - Decodificar Morse digitado
- 3 - Codificar arquivo de texto

- 4 - Decodificar arquivo Morse
- 5 - Mostrar árvore
- 0 - Encerrar

Flood Fill

- 1 - Executar com pilha
- 2 - Executar com fila
- 3 - Escolher imagem
- 4 - Escolher coordenada de início .:(exemplo – digite X, <lê> , digite Y <lê> ... volta para menu)
- 0 - Encerrar

O programa deve validar opções, coordenadas, arquivos inexistentes e entradas inválidas, exibindo mensagens compreensíveis.

6 Recursos de apoio

- Vídeo sobre árvore de Código Morse: youtube.com/watch?v=BqeX-4TshEA.
- Documentação oficial do Java para BufferedImage e ImageIO.
- Documentação sobre leitura de arquivos e tratamento de exceções.
- Imagens com regiões de cores sólidas para testar o Flood Fill.

Materiais externos e ferramentas de IA podem auxiliar. A equipe deve informar as referências, compreender o código incorporado e conseguir explicá-lo e modificá-lo durante a defesa.

7 Entrega

A equipe deverá entregar no AVA um arquivo ZIP contendo duas pastas, cada uma correspondente a um dos repositórios GitHub.

- um repositório exclusivo para o projeto Código Morse;
- um repositório exclusivo para o projeto Flood Fill, contendo as versões com pilha e com fila;
- **os dois repositórios compartilhados com o usuário profeara;**
- um ZIP contendo exatamente duas pastas: CódigoMorse e FloodFill;
- cada pasta com código-fonte funcional, README e arquivos usados nos testes.

O README de cada projeto deverá explicar como compilar e executar a aplicação. O ZIP enviado ao AVA será a cópia oficial e imutável da entrega utilizada caso haja divergência com os repositórios.

7.1 Prazo e congelamento dos repositórios

O prazo final será **08/10/2026**. Os repositórios não deverão receber alterações depois dessa data. Alterações posteriores resultarão em **desconto de 10%** na nota. O professor poderá

8 Critérios de avaliação

Componente	Percentual	O que será observado
Código Morse	35%	Árvore, codificação, decodificação, arquivo e visualização.
Flood Fill	35%	Versões com pilha e fila próprias, processamento e imagens das etapas.
Organização e entrega	15%	Orientação a objetos, dois repositórios, ZIP e READMEs.
Defesa e autoria	15%	Domínio do código e

		alteração solicitada.
Total	100%	

9 Defesa e prova de autoria

No dia da avaliação, os projetos serão baixados (**DO AVA**) na presença do professor. A equipe deverá compilar e executar o Código Morse e as duas versões do Flood Fill, demonstrar as funcionalidades e explicar as estruturas utilizadas. O professor poderá solicitar uma pequena alteração no código ou nos dados de entrada. O desempenho será individual, mesmo quando o trabalho for realizado em equipe.

O estudante que não comparecer à defesa, sem justificativa formal aceita pela instituição, terá nota zero no TDE. A incapacidade de explicar ou modificar o código poderá reduzir ou anular a nota individual.

10 Integridade acadêmica

Cópias não identificadas, códigos iguais entre equipes ou soluções que os integrantes não consigam explicar serão analisados como possível plágio. Na defesa será utilizada exclusivamente a versão registrada dentro do prazo.

Verificação final. Antes de enviar, faça um novo clone de cada repositório, siga apenas as instruções dos READMEs e confirme que o Código Morse, o Flood Fill com pilha e o Flood Fill com fila compilam e executam corretamente.